Lab Report: Tracing a Response from w.Write() to the Wire

Names: Ian Burns, Bryan Hernandez

Course: Systems & Network Programming in Go

https://github.com/ianburns17/lab_4

Topic: HTTP response lifecycle, buffering, and kernel writes
Environment: Go 1.22+ (tested with Go 1.26.6 on Windows/Linux)

Purpose

This lab follows a response from a Go handler to the client. It focuses on the two Go buffers, the kernel socket buffer, and the three different moments when response headers are fixed, built, and sent.

Response path and header moments

1. **Buffer A — 2,048 bytes:** request-owned buffer that collects handler writes.
2. **chunkWriter — no buffer:** builds the HTTP headers when Buffer A first empties.
3. **Buffer B — 4,096 bytes:** connection-owned buffer that batches output before socket writes.
4. **Kernel socket send buffer:** operating-system buffer for the TCP connection.

Buffer A empties when it fills, the handler flushes, or the handler finishes. Buffer B empties when it fills, a flush occurs, or the request finishes.

- **Moment 1 — status fixed:** the first WriteHeader or Write fixes the status. If the handler calls Write first, Go defaults to 200 and snapshots the header map.
- **Moment 2 — header built:** when Buffer A first empties, chunkWriter builds the header and chooses Content-Length or chunked encoding. It also sniffs Content-Type if none was set.
- **Moment 3 — socket write:** Buffer B sends bytes to the kernel. The client cannot receive the response before this point.

Experiment 1: One small response

The /hello handler wrote 26 bytes. The handler returned before the socket write, and the log showed one conn.Write of 143 bytes. The response used Content-Length: 26.

Questions and answers

1. **Why Content-Length: 26 instead of chunked encoding?** The body fit in Buffer A and the handler finished before the buffer had to empty. Go knew the complete body length when it built the headers.

2. **Which log lines show Moments 1 and 3? Why is Moment 2 missing?** Moment 1 is HANDLER first Write: net/http calls WriteHeader(200) for you; status fixed here. Moment 3 is the SOCKET conn.Write: 143 bytes... line. Moment 2 happens inside net/http, between the instrumented ResponseWriter and net.Conn boundaries, so the lab does not log it directly.
3. **Where were the 26 bytes when the handler returned, and what does "I already wrote the response" mean?** They were still in Buffer A. Calling w.Write() does not necessarily send bytes to the client; the data can remain in user-space buffers until a flush or request completion.

Experiment 2: Watching the buffers fill

For /drip?size=1000&count=8&pause=300, Write #3 pushed Buffer A beyond 2,048 bytes. Buffer B filled around Write #5, and the first socket write occurred about 1.2–1.4 seconds after the request began. The response used chunked encoding because the handler was still running when Buffer A first emptied.

With 100-byte writes, the total body was only 800 bytes. Neither buffer filled, so the client received the response after the handler returned, about 2.4 seconds later. Go could use Content-Length: 800.

Questions and answers

1. **Why did the 1,000-byte writes use chunked encoding while the 100-byte writes used Content-Length?** The 1,000-byte run forced Buffer A to empty while the handler was still running, so Go could not know the final body size. The 800-byte body stayed in Buffer A until the handler finished, making its length known.
2. **Why was time to first byte different even though both handlers took about the same time?** The larger writes filled both buffers during the handler, causing an early socket write. The smaller writes filled neither buffer, so the response waited until handler completion.
3. **A page takes three seconds to build and writes small pieces without flushing. What holds the data, and what could the developer do?** The bytes remain in Buffer A, or may reach Buffer B without filling it. The developer can flush after writing critical initial content, or write enough data to fill the buffers. Explicit flushing is the more intentional option when early output matters.

Experiment 3: Many small writes versus one large write

Endpoint	Handler writes	Body size	Socket writes	Header
/pieces?n=100	100	1,000 B	1 × 1,119 B	Content-Length: 1000
/pieces?n=300	300	3,000 B	1 × 3,144 B	chunked
/pieces?n=1000	1,000	10,000 B	4,096 B, 4,096 B, 1,973 B	chunked

/big?kb=1	1	1,024 B	1 × 1,143 B	Content-Length: 1024
/big?kb=64	1	65,536 B	4,096 B, 61,572 B, 7 B	chunked

For /pieces?n=300, Buffer A overflowed during the handler, so Go built a chunked header. The combined output still fit in Buffer B, so the socket write waited until the handler returned. This separates Moment 2 from Moment 3. For /big?kb=64, Go sent an initial buffered block, then wrote the large remainder directly, followed by the 7-byte chunk terminator.

Questions and answers

1. **Where were the bytes when 1,000 handler writes resulted in only three socket writes?** The writes were accumulated and coalesced in Buffer A and Buffer B before reaching the kernel.
2. **Why did /pieces?n=300 build its header during the handler but log a socket write after the handler returned?** The 3,000-byte body overflowed Buffer A, triggering header construction. The resulting output was still under Buffer B's 4,096-byte capacity, so Buffer B waited until request completion to write to the socket.
3. **Is joining small strings necessary to reduce system calls? When might it still help?** Not usually just to reduce socket writes, because Go already batches small writes. Building the response first can still help reduce per-write overhead, calculate a length in advance, or avoid sending partial output if generation fails.

Experiment 4: Flush()

For /hold?pause=1500, the no-flush response reached the client after about 1.71 seconds. With flushing enabled, the first socket write happened before the 1.5-second sleep and time to first byte was about 0.20 seconds. The flushed response used chunked encoding; the final write sent the 0\r\n\r\n terminator.

Questions and answers

1. **Does Flush() empty A, B, or both?** It forces the pending response through Buffer A and Buffer B to the connection, as shown by the socket write immediately after Flush() requested.
2. **Why did the header change from Content-Length to chunked encoding, and which moment decides?** Moment 2. Flushing while the handler is still running means Go cannot know whether more body data will follow, so it uses chunked encoding.
3. **When is a developer right that a fast handler does not need to flush?** When the handler can finish promptly and the client does not need partial output before completion. Automatic flushing at the end lets Go send the completed response together and may preserve Content-Length.

Experiment 5: Headers and errors after the first write

Late header: The handler set X-Before, wrote five bytes, and then set X-After. The client received X-Before but not X-After. Moment 1 had already occurred, so the later change did not affect the header snapshot.

Late error: The handler wrote part of an HTML page and then called http.Error(..., 500). The client still received 200 OK; the error text was appended to the partial HTML, and Go logged a superfluous WriteHeader warning. The first write had already fixed the status.

Safe pattern: render into a bytes.Buffer before writing to w. If rendering fails, send the error before Moment 1. If it succeeds, set the content type and write the completed result.

```
func reportPage(w http.ResponseWriter, r *http.Request) {

   tmpl, err := template.ParseFiles("report.html")

   if err != nil {

      http.Error(w, "Internal Server Error", http.StatusInternalServerError)

      return

   }



   var buf bytes.Buffer

   if err := tmpl.Execute(&buf, data); err != nil {

      http.Error(w, "Internal Server Error", http.StatusInternalServerError)

      return

   }



   w.Header().Set("Content-Type", "text/html; charset=utf-8")

   buf.WriteTo(w)

}
```

Questions and answers

1. **Why was setting X-After too late even though the client had received no bytes?** Moment 1 had already happened and Go had taken its header snapshot. The data did not need to reach the client for the status and headers to become fixed.
2. **At what moment did the status become 200, and which line shows it?** Moment 1, shown by HANDLER first Write: net/http calls WriteHeader(200) for you; status fixed here.
3. **Why does the safe handler return the right status?** Rendering into a private buffer does not touch w. If an error occurs, http.Error is the first operation on w, so it can set status 500. In the late-error case, writing the page first already committed status 200.

4. **Why is buffering a 500 MB download a bad idea, and what should be checked first?** It could use hundreds of megabytes of RAM per request. Before writing, check that the file exists, permissions and authorization are valid, and any other likely preconditions pass. Then set headers and stream with io.Copy. An error after streaming starts cannot be turned into a clean 500 response.

Experiment 6: Kernel writes and backpressure (optional)

A Linux strace line such as write(..., 143) = 143 means the kernel accepted 143 bytes from the process. It does not prove that the client received all of them.

When a client stops reading, its receive window can shrink to zero. The server's kernel send buffer may eventually fill, causing a socket write—and therefore the handler's w.Write()—to block until space becomes available or the connection closes.

Questions and answers

1. **What does the number after = mean? Why does = 143 not prove delivery?** It is the return value of the system call: the kernel accepted 143 bytes. Those bytes may still be queued in the local TCP stack and may not have been acknowledged by the client.
2. **Why did the handler's write block for eight seconds? Which buffer was full?** The kernel socket send buffer filled because the client stopped draining the connection. The kernel could not accept more data until space became available.
3. **Is the kernel send buffer per request or per connection? What does that mean for keep-alive?** It is per TCP connection. HTTP/1.1 keep-alive requests reuse that connection, so a stalled connection can delay subsequent responses on it.

Exercise A: /drip?size=700&count=5&pause=300

Five writes produce 3,500 body bytes. Buffer A overflows on Write #3, causing the header to be built while the handler is running. Buffer B does not fill, so the socket write occurs after the handler returns. The observed socket write was 3,644 bytes and the response used chunked encoding.

Questions and answers

1. **How many socket lines appear, and when?** One, after the handler returns, because Buffer B never reaches 4,096 bytes during the handler.
2. **Will the header use Content-Length or chunked encoding?** Chunked, because Buffer A first empties while the handler is still running.
3. **What verified the prediction?** The log showed the handler returning after five writes totaling 3,500 body bytes, followed by one 3,644-byte socket write. The remaining bytes are headers and chunk framing.

Exercise B: /drip?size=1500&count=6&pause=300

Six writes produce 9,000 body bytes.

1. **When does the first socket write appear?** On Write #3, when Buffer B exceeds 4,096 bytes.
2. **Is there another socket write during the handler?** Yes, predicted on Write #5 when Buffer B fills again.

3. **How many socket writes occur, and how large is the last?** Three: 4,096 bytes, 4,096 bytes, and about 973 bytes after the handler finishes. Together they account for the 9,000-byte body plus headers and chunk framing.

Exercise C: Architecture summary

1. Handler: Calls w.Write()

2. Moment 1: Response Committed
- Status code is finalized.
- Headers are committed.

3. Buffer A — 2,048 bytes (per request)
- Flushes when full, explicitly flushed, or the handler finishes.

4. chunkWriter
- Builds response headers.
- Selects Content-Length or chunked transfer encoding as needed.

5. Buffer B — 4,096 bytes (per TCP connection)
- Flushes when full, explicitly flushed, or the request finishes.

6. Moment 3: net.Conn.Write()
- Passes bytes to the operating system.

7. Kernel Socket Send Buffer
- OS-managed buffer for the TCP connection.

8. TCP/IP Network
- Transmits data toward the client.

9. Client
- Receives the HTTP response.

Reflection questions

1. **How many buffers hold response bytes between w.Write() and the network? Give sizes and owners.** Buffer A is 2,048 bytes and belongs to a request; Buffer B is 4,096 bytes and belongs to a TCP connection; the kernel send buffer is OS-managed and belongs to the TCP socket. chunkWriter is a decision/formatting component, not another holding buffer.
2. **What empties Buffer A and Buffer B, and how do they differ?** A empties when it fills, the handler flushes, or the handler finishes. B empties when it fills, a flush reaches it, or the request finishes. A's first empty triggers header construction (Moment 2); B's first empty triggers the socket write (Moment 3). A is per request; B is reused with the connection.
3. **Explain Moments 1, 2, and 3 using the logs.** In Experiment 1, the first HANDLER first Write line shows Moment 1. In /pieces?n=300, Buffer A overflowed during the handler and caused Moment 2, but no socket line appeared until handler completion because Buffer B was not full. The later SOCKET conn.Write line shows Moment 3.
4. **A handler writes 100 bytes and then runs a five-second database query. What does the client see, what line changes it, and what is the cost?** Without a flush, the client sees no response bytes during the query. Calling http.NewResponseController(w).Flush() after the write sends the pending response earlier.

The cost is that Go may need chunked encoding because the handler is still running, and it may perform extra socket writes rather than sending the completed response as one batch.

5. **Why can a handler not report a failure halfway through a response? What is the safe pattern?** The first write fixes the status and headers. A later http.Error cannot replace a committed 200 response and may only append error text. Render into a buffer first; send an error if rendering fails, and write the buffered response only after success.
6. **Why can a slow client keep the handler from returning?** The client stops reading and its receive window may close. The server's kernel send buffer fills, so a later socket write blocks until the client reads again or the connection closes.
7. **Which experiment changed the mental model most, and what was the earlier assumption?** Experiment 3, especially /pieces?n=300. It shows that building headers does not mean the bytes have already been sent. Moment 2 can happen while output remains in Buffer B, and Moment 3 can happen later.
8. **What question remains about HTTP/2 and HTTP/3?** How does this pipeline change when HTTP/2 uses HEADERS and DATA frames and multiplexed streams, or HTTP/3 uses QUIC, instead of HTTP/1.1 chunked transfer encoding? In HTTP/2, connection output is managed through frame queues and flow-control windows rather than the same HTTP/1.1 chunked-body path.

Conclusion

Calling w.Write() does not necessarily send data immediately. Buffer A determines when Go needs to build the headers, Buffer B batches bytes before socket writes, and the kernel buffer handles outgoing TCP data. Flush() can make data available sooner, but it may change transfer framing and add writes. Once the status and headers are committed, they cannot be replaced, so responses that may fail during generation should be prepared before writing to http.ResponseWriter.